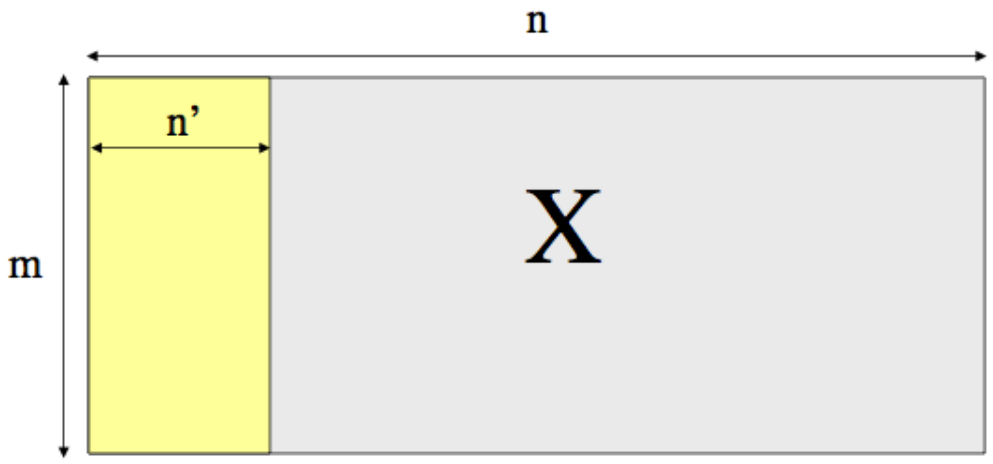
Feature Selection

Why Feature Selection?

- High-dimensional data is everywhere:
  - Text: >100,000 features
  - Bioinformatics: thousands of genes
  - Images: thousands of pixels
- Goal: Build models that are:
  - **better,**
  - **faster, and**
  - **more interpretable.**



Problems with High Dimensions

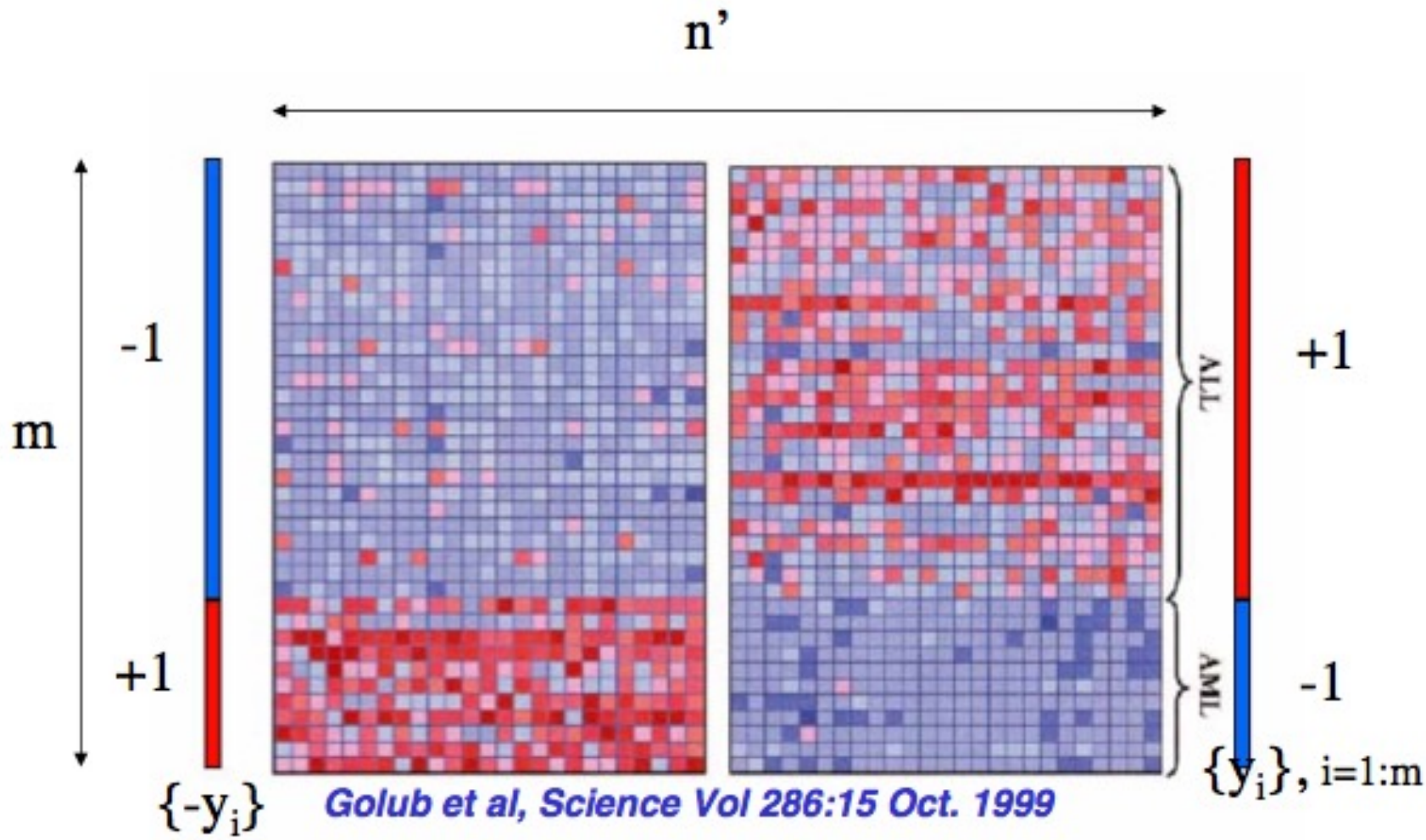
- Challenges:
  - Overfitting (too many features, not enough data)
  - Noise from irrelevant features
  - Redundant features
  - Computational cost

- Special case: n >> m problem
  - #features >> #samples

Curse of Dimensionality

- Phenomena arises when analyzing and organizing data in high dimensional spaces that do not occur in low dimensional settings
  - Data becomes sparse
    - Difficult to have enough data to obtain and support results that are statistically sound and reliable
  - Distances become less meaningful
    - All objects appear to be sparse and dissimilar making it difficult to detect areas where objects form groups of similar properties
  - Hard to detect clusters/patterns
- Hughes phenomenon:
  - More features → worse performance (fixed data)

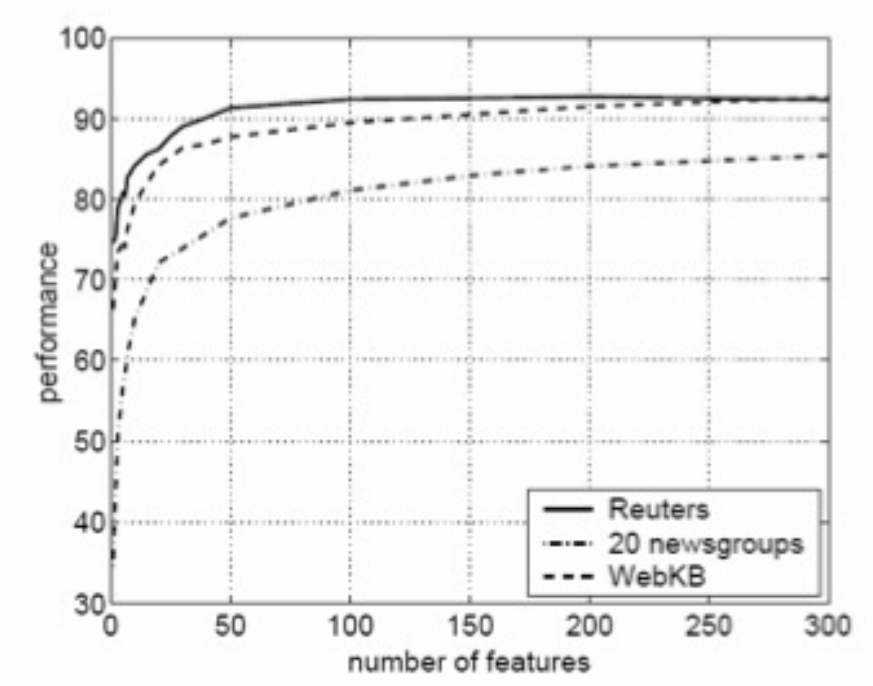
Example 1: Gene Selection



ALL – positive class, Acute lymphoblastic leukemia
AML -- Acute myeloid leukemia

Example 2: Text Data

- Reuters: 21578 news wires, 114 semantic categories
- 20 newsgroups: 19997 articles, 20 categories
- WebKB: 8282 web pages, 7 categories
- Bag-of-words: > 100000 features



- Top 3 words of some categories:
  - **Alt.atheism: atheism, atheists, morality**
  - **Comp.graphics: image, jpeg, graphics**
  - **Sci.space: space, nasa, orbit**
  - **Soc.religion.christian: god, church, sin**
  - **Talk.politics.mideast: israel, armenian, turkish**
  - **Talk.religion.misc: jesus, god, jehovah**

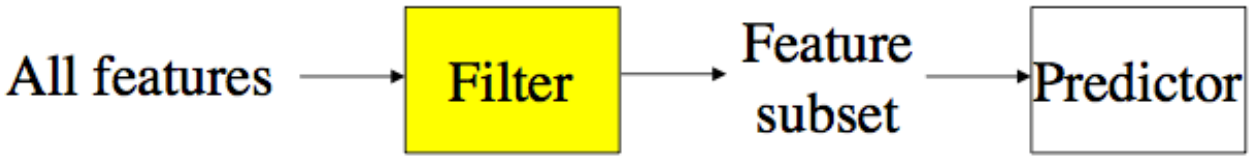
What is a “Good Feature”?

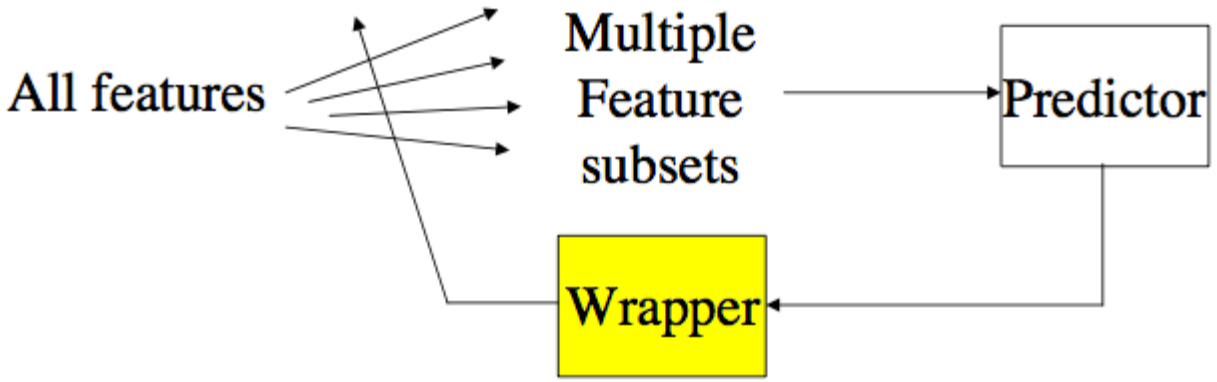
- A good feature should be:
  - **Relevant** → related to target Y
  - **Non-redundant** → not duplicate info
  - **Useful in combination** → interaction matters

Types of Feature Selection Methods

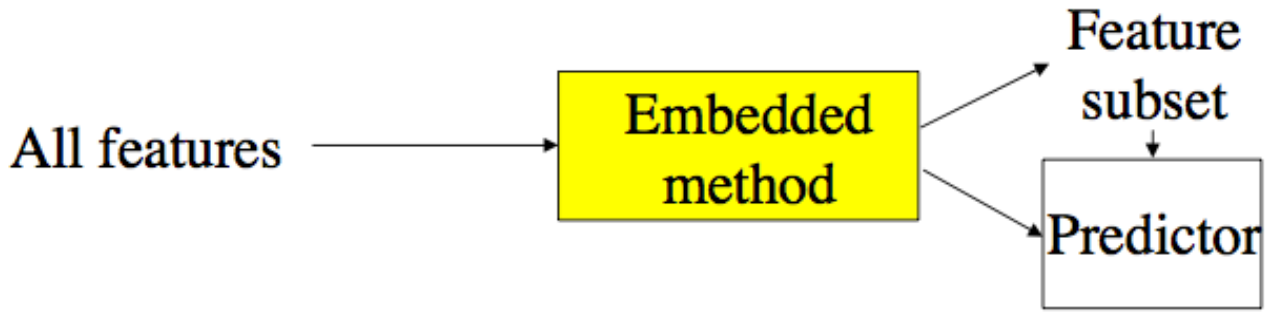
- Filter methods
  - independent of model
- Wrapper methods
  - use model performance
- Embedded methods
  - built into model training

Filters, Wrappers and Embedded methods





Key difference:
- Filter → before model
- Wrapper → around model
- Embedded → inside model



UNIVARIATE METHODS

Univariate Feature Selection

- Evaluate **one feature at a time**
- Ignore interactions

- Simple and fast

Methods Overview

Method	Idea	Use case
Variance	remove low variation	unsupervised
Correlation	linear relation	regression
T-test	mean difference	binary class
Chi-square	dependence	categorical
Mutual Info	general dependence	any

Variance Threshold

- Remove features with low variance
  - Assuming features with larger variance contain higher information
- Intuition: constant features → useless
  - Compute the variance of all the features, and eliminate features having variance lower than a predefined threshold value, or
  - Keep only the top k features having the largest variance values

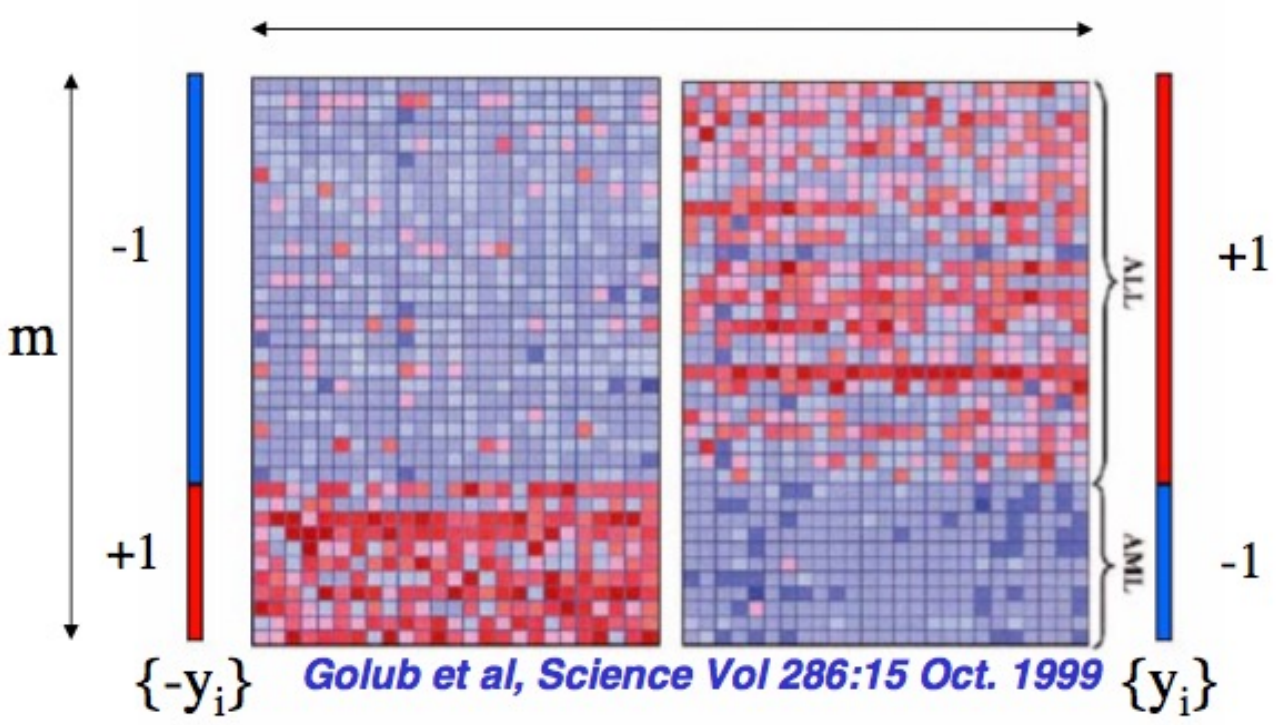
Correlation

- Measure linear relationship with target
  - Measure linear relationship with target
  - Low correlation → weak predictor
  - Limitation: only captures **linear** relationship
- Standardize/normalize data

$$x_i = \frac{x_i - \mu}{\sigma} \qquad \rho_{X,Y} = \frac{\operatorname{cov}(X,Y)}{\sigma_X \sigma_Y} = \frac{E[(X - \mu_X)(Y - \mu_Y)]}{\sigma_X \sigma_Y},$$

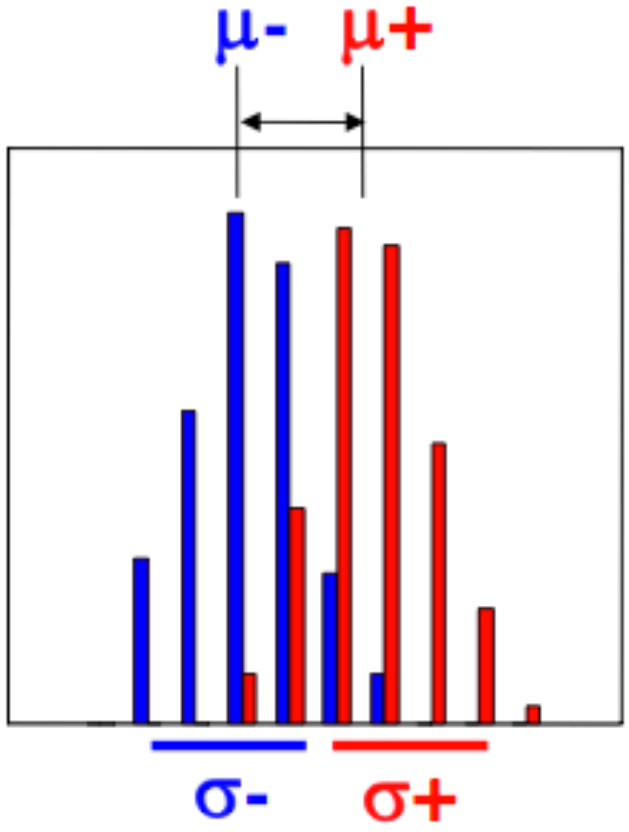
  - (μ and σ are the mean and standard deviation of the attribute)
  - Compute Pearson correlation between x_i and Y

Signal to Noise Ratio (S_2N)



$$S_2N = \frac{|\mu^+ - \mu^-|}{\sigma^+ + \sigma^-}$$

Higher S_2N → more predictive the feature is to the target variable Y

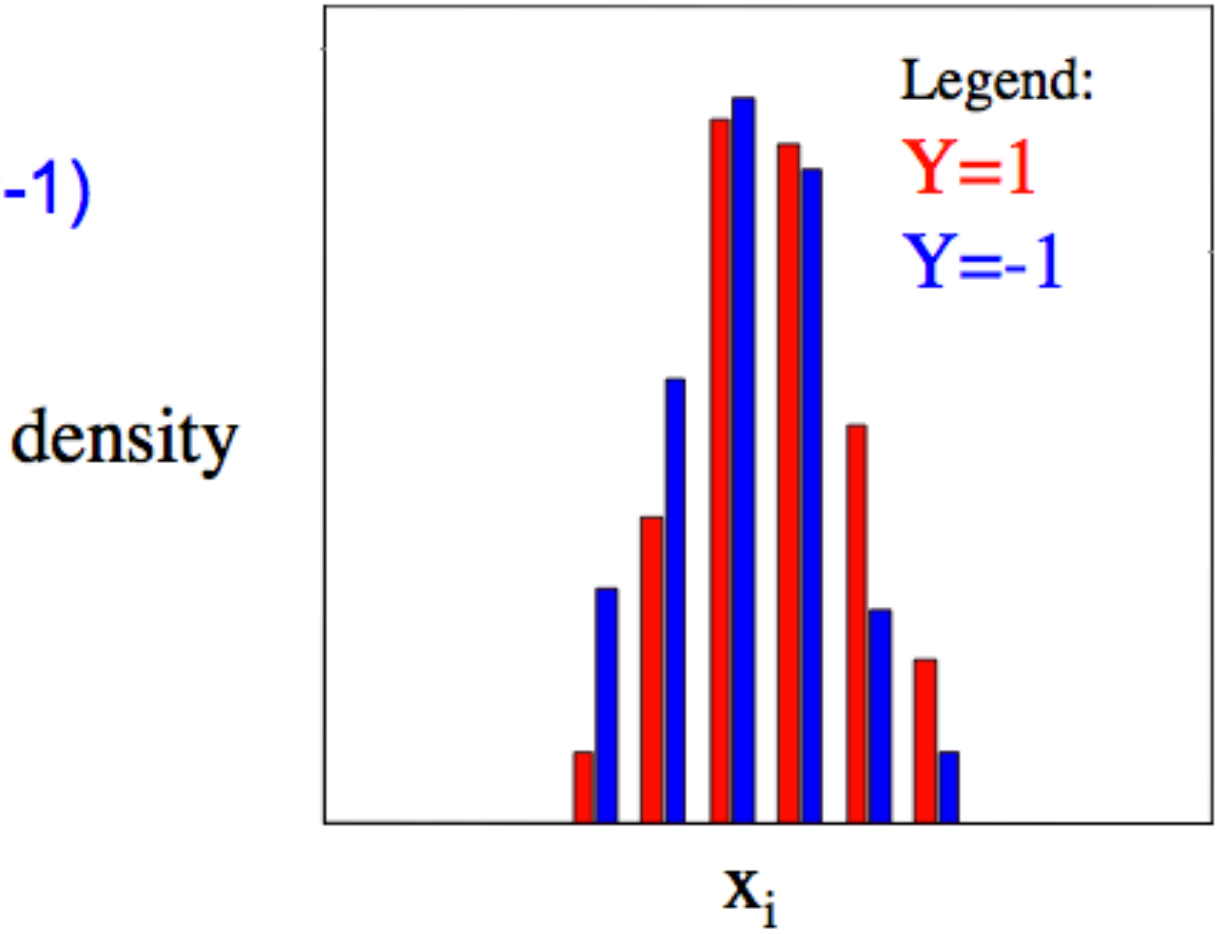


Individual Feature Irrelevance

$P(X_i, Y) = P(X_i)\, P(Y)$

$P(X_i | Y) = P(X_i)$

$P(X_i | Y=1) = P(X_i | Y=-1)$



Practice Question

Obj	A_1	A_2	A_3	…	class
1	30	28	.		Pos
2	24	16	.		Pos
3	20	15	.		Neg
4	28	17	.		Pos
5	10	19	.		Neg
6	20	20	.		Neg
7	16	16	.		Neg
8	34	15	.		Pos
.	.	.	.		.
.	.	.	.		.

Which attribute is better for predicting the class label?

A_1 or A_2 ?

Practice Question (cont.)

Obj	A_1	A_2	Class
1	30	28	positive
2	24	16	
4	28	17	
8	34	15	

($\mu_1 = 29$, $\sigma_1 = 4$) ($\mu_2 = 19$, $\sigma_2 = 5$)

$$S2N_{A1} = \frac{|29 - 16.5|}{4 + 4} = \frac{12.5}{8} = 1.5625$$

Obj	A_1	A_2	Class
3	20	15	negative
5	10	19	
6	20	20	
7	16	16	

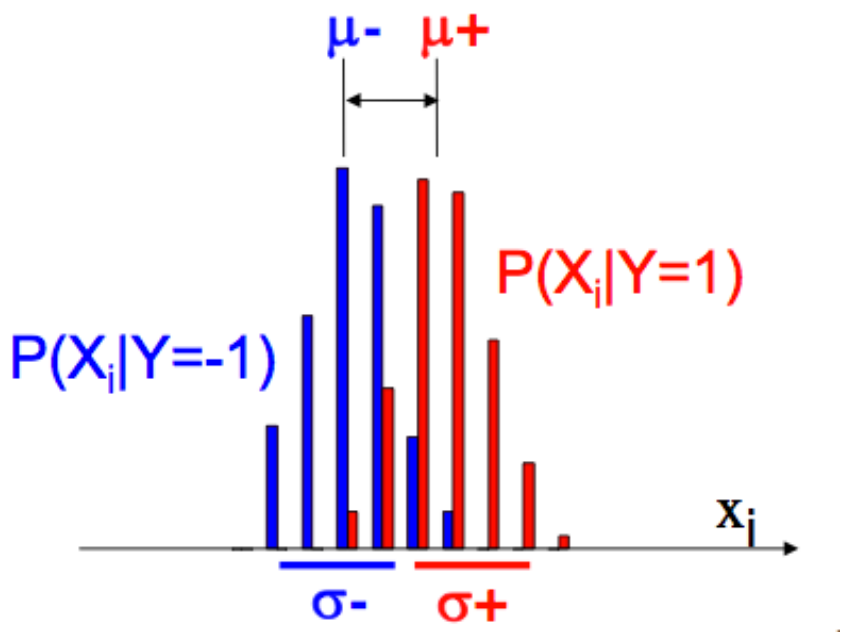
($\mu_1 = 16.5$, $\sigma_1 = 4$) ($\mu_2 = 17.5$, $\sigma_2 = 5$)

$$S2N_{A2} = \frac{|19 - 17.5|}{5 + 5} = \frac{1.5}{10} = 0.15$$

Feature A1 is much better than A2

T-test (two tailed)

- Compare means of two classes
- Null hypothesis H_0: $\mu^+ = \mu^-$
  - If rejected → feature is useful



- T statistic: If H_0 is true,

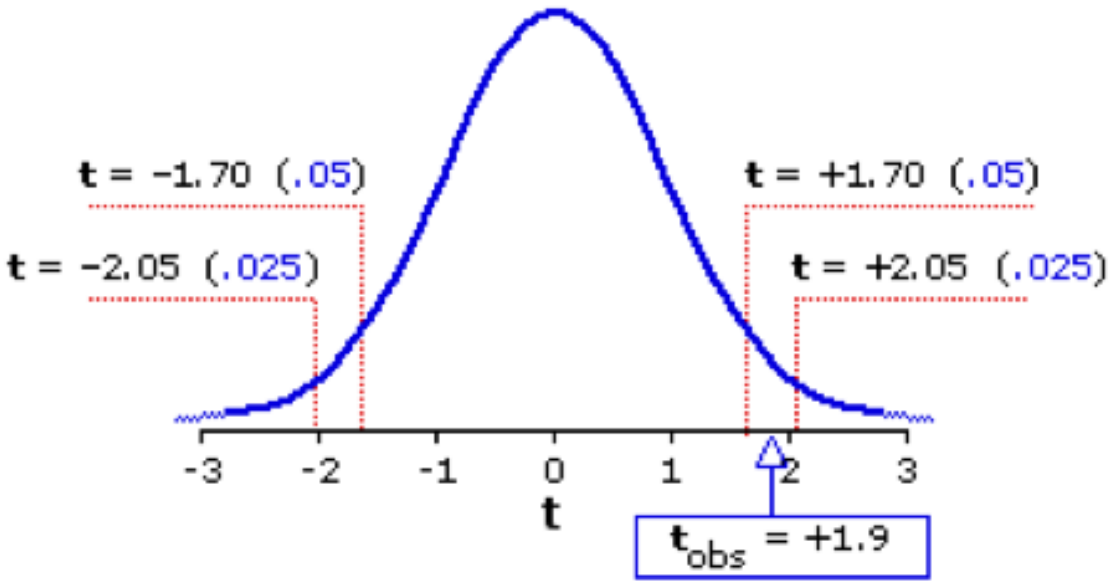
$$t = \frac{(\mu^+ - \mu^-)}{\sigma_{within}\sqrt{\frac{1}{m^+} + \frac{1}{m^-}}},$$

m^+ and m^- are the numbers of rows in Y = 1 and Y = -1 classes

- Degree of freedom: $m^+ + m^- - 2$
- Confidence, e.g., 95%

Normally distributed classes,
equal variance σ^2 unknown;
estimated from data as σ^2_{within}.

T-test



Level of Significance for a Directional Test				
.05	.025	.01	.005	.0005
Level of Significance for a Non-Directional Test				
---	.05	.02	.01	.001
df = 28	1.70	2.05	2.47	2.76	3.67

Procedure

Two t-values:

- t-value calculated from the observations
- Critical t-value

If $t_{obs} > t_{critical}$,
reject the hypothesis
Otherwise,
accept the hypothesis

Null hypothesis H_0: $\mu^+ = \mu^-$

Practice Question

Obj	A_1	A_2	A_3	…	class
1	30	28	.		Pos
2	24	16	.		Pos
3	20	15	.		Neg
4	28	17	.		Pos
5	10	19	.		Neg
6	20	20	.		Neg
7	16	16	.		Neg
8	34	15	.		Pos
.	.	.	.		.
.	.	.	.		.

Should attributes A_1 or A_2 be eliminated when predicting for the class label? Apply two tailed T-test for determination, set confidence level at 95%.

Practice Question (cont.)

Obj	A_1	A_2	Class
1	30	28	positive
2	24	16	
4	28	17	
8	34	15	

($\mu_1 = 29, \sigma_1 = 4$) ($\mu_2 = 19, \sigma_2 = 5$)

Obj	A_1	A_2	Class
3	20	15	negative
5	10	19	
6	20	20	
7	16	16	

($\mu_1 = 16.5, \sigma_1 = 4$) ($\mu_2 = 17.5, \sigma_2 = 5$)

$m^+ = m^- = 4$

$\sqrt{\frac{1}{4} + \frac{1}{4}} = \sqrt{0.5} \approx 0.707$

Degrees of freedom: $4 + 4 - 2 = 6$

At 95% confidence (two-tailed):

$t_{\text{critical}} \approx 2.447$

Feature A1

$\mu^+ = 29, \mu^- = 16.5$

Difference = **12.5**

$\sigma_{\text{within}} \approx 4$

$$t_{A1} = \frac{12.5}{4 \times 0.707} \approx \frac{12.5}{2.828} \approx 4.42$$

4.42>2.447 significant

Feature A2

$\mu^+ = 19, \mu^- = 17.5$

Difference = **1.5**

$\sigma_{\text{within}} \approx 5$

$$t_{A2} = \frac{1.5}{5 \times 0.707} \approx \frac{1.5}{3.536} \approx 0.42$$

0.42<2.447 not significant

Feature A1 is much better than A2

Chi-Square Test

- Measures **dependence** between feature and class
  - If independent → useless
  - If dependent → useful

Chi-Squared Test Example

- Question: Can **Gender** predict **Footwear Preference**?
- Data:

	Sandals	Sneakers	Leather	Boots	Other	Total
Male	6	17	13	9	5	50
Female	13	5	7	16	9	50
Total	19	22	20	25	14	100

Key Idea

- What are we testing?
  - Null hypothesis:
    H_0:Gender and Footwear are independent
  - If true:
    - Gender gives **NO information** → useless feature
  - If false:
    - Gender helps prediction

- Chi-Square Formula $\chi^2 = \sum \frac{(O-E)^2}{E}$
  - how far observed values are from expected values

Expected Counts

$$E = \frac{(\text{row total})(\text{column total})}{\text{grand total}}$$

$$E_{\text{Male, Sandals}} = \frac{50 \times 19}{100} = 9.5$$

	Sandals	Sneakers	Leather shoes	Boots	Other	Total
Male Observed	6	17	13	9	5	50
Male Expected	9.5	11	10	12.5	7	
Female Observed	13	5	7	16	9	50
Female Expected	9.5	11	10	12.5	7	
Total	19	22	20	25	14	100

Result

- From computation: $\chi^2 = 14.026$

- Degrees of freedom: (2−1)(5−1)=4
- Critical value (95%): 9.49

- Decision:

14.026>9.49

Reject H_0

Interpretation

- What does this mean?
  - Gender and footwear are **NOT independent**
  - Gender provides **useful information**

Therefore

Gender is a GOOD feature in predicting footwear choice

Mutual Information (Concept + Definition)

- Goal: Measure how much information a feature X provides about the target Y
- Key Idea:
  - MI(X;Y)=0 if and only if X and Y are independent
  - Larger MI → stronger dependency → more useful feature

- Definition (Discrete Case)

$$MI(X;Y) = \sum_x \sum_y P(x,y) \log_2 \frac{P(x,y)}{P(x)P(y)}$$

P(x,y): joint probability, P(x), $P(y)$: marginal probabilities

- Interpretation
  - If $P(x,y) = P(x)P(y)$→ x and y are independent → MI = 0
  - If very different → strong dependence → MI large
- MI captures **nonlinear relationships**

Mutual Information (Connection to Entropy)

- Entropy $H(Y) = -\sum_{y} P(y) \log_2 P(y)$
  - Measures **uncertainty** in Y

- Conditional Entropy $H(Y|X) = -\sum_{x} P(x) \sum_{y} P(y|x) \log_2 P(y|x)$
  - Measure **remaining uncertainty** after observing X
- Mutual Information: $MI(X;Y) = H(Y) - H(Y|X)$
  - Reduction in uncertainty of Y after observing X
  - Large MI → knowing X greatly reduces uncertainty
  - Small MI → X provides little information

MI for Feature Selection

- How we use MI
  - Rank features by: $MI(X_i; Y)$
  - Select top-k features

- Limitation
  - Still **univariate**
  - Does NOT handle redundancy

Mutual Information for continuous features

- For feature X and target variable Y:

$$P(x,y) = \text{joint probability of } (x,y), \quad x \in \{1,\ldots,r\} \text{ and } y \in \{1,\ldots,s\}$$

$$P(x) = \int_y P(x,y)dy = \text{marginal probability of } x$$

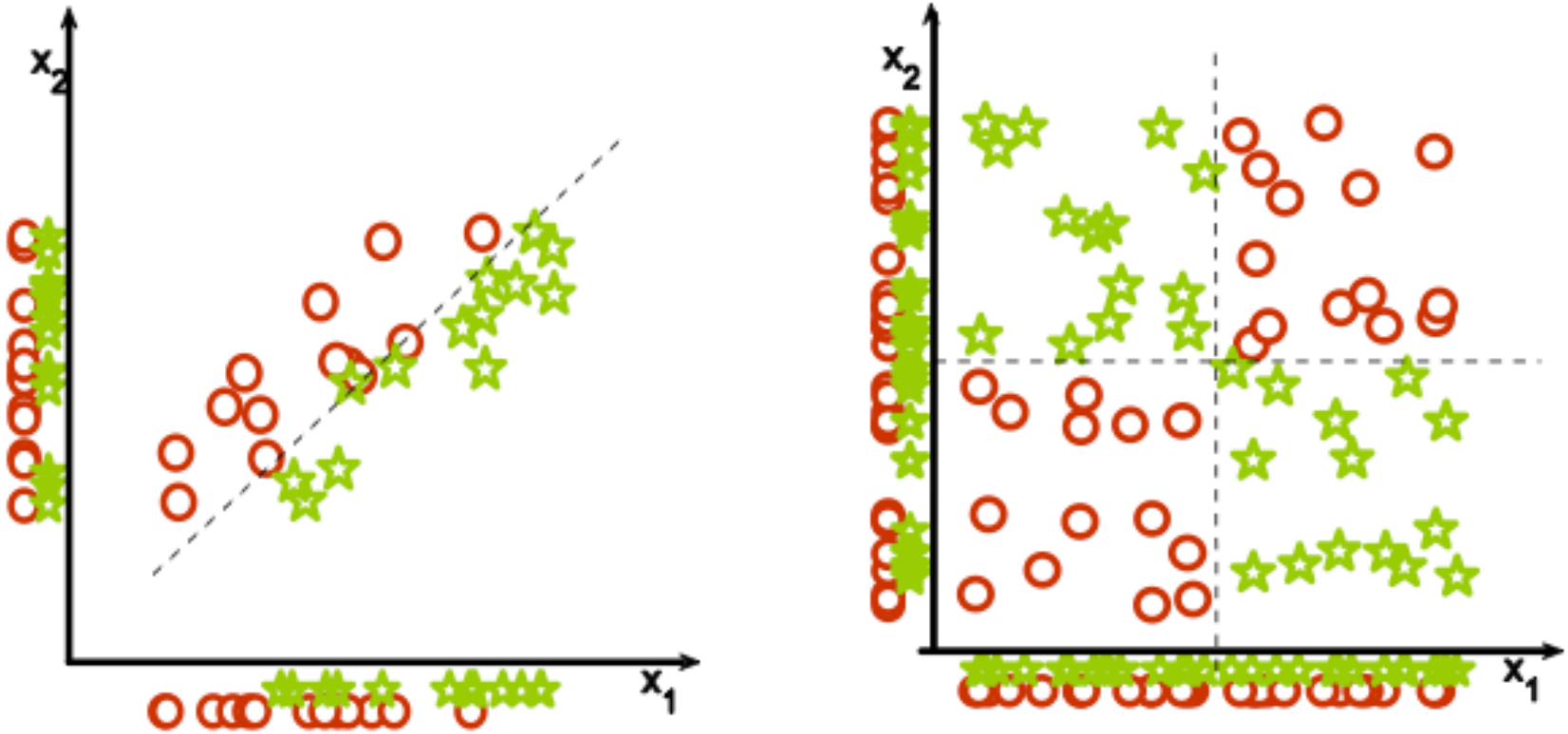
$$P(y) = \int_x P(x,y)dx = \text{marginal probability of } y$$

- Mutual Information:

$$0 \le MI(X;Y) = \iint_{x,y} P(x,y)\log\frac{P(x,y)}{P(x)P(y)}dxdy$$

MULTIVARIATE METHODS

Univariate May Fail



Why Multivariate?

- Consider feature interactions
- Reduce redundancy
- Two approaches:
  - Feature Selection with Mutual Information (MIFS)
  - Relief

Feature Selection with Mutual Information

- Key idea:
  - Select features that:
    - are informative about Y
    - but NOT redundant with selected features
- Approach:
  - Given all features F, select a subset S of k features that provides the **most information about the class** C (i.e., maximizes $I(C; S)$or minimizes $H(C \mid S)$

MIFS Algorithm

- Basic Idea:
  - Given a set of already selected features, the algorithm chooses the next feature as the one that maximizes the information about the class corrected by subtracting a quantity proportional to the average MI with the selected features.
  - In order to be selected, a feature must be informative about the class without being predictable from the current (chosen) set of features.
    - if two features ***f and f' are highly dependent, I (f ; f ')will be large and, after the better one is picked, the selection of the second one is penalized.***

MIFS Algorithm

Step 1: Initialization: F={initial set of n features}; S={}

Step 2: Computation of the MI with the output class

for each feature *f* compute I(C; *f*).

Step 3: Choice of the first feature

find the feature f that maximizes I(C, *f*): F = F – {*f*}; S = S + {*f*}

Step 4: Greedy Search:

Repeat until |S|=k:

(a) Computation of MI between features

for all pairs of features (*f*, s) where *f* is in F, and s is in S,

compute I(*f*; s)

(b) Selection of the next feature

choose feature f as the one that maximizes $I(C;f) - \beta \sum_{s \in S} I(f;s)$

F = F – {*f*}; S = S + {*f*}

Step 5: Output the set S containing the selected features

Relief: Core Idea

- Relief Algorithm
  - Does a feature help distinguish between classes?
    - Features that separate different classes → **important**
    - Vary within same class → **not useful**
  - A feature gets **high weight** if:
    - Across MANY data points:
      - small differences within same class
      - large differences across classes
  - A feature gets **low / negative weight** if:
      - large variation within same class
      - little difference across classes

Relief Algorithm Intuition

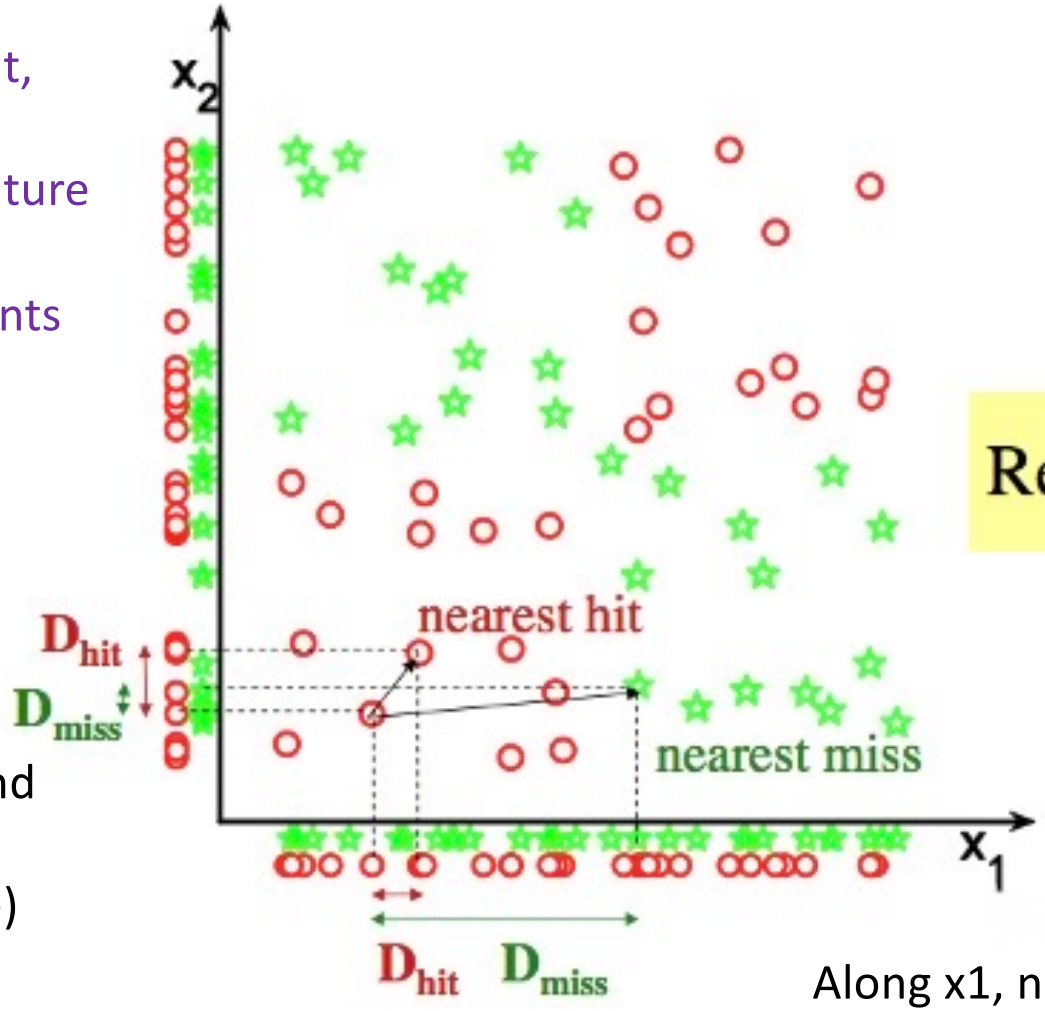
- For each randomly selected instance X:
  - Find:
    - Near Hit (same class)
    - Near Miss (different class)
  - Update weights:
    - Wf=Wf – diff to hit + diff to miss
- Then:
  - Repeat for many samples (often all or many random picks)
  - Final weight = **average effect over all points**

Relief

For each data point:

- Determine near hit, and near miss
- Determine the feature weight update

Repeat for MANY points



$$Relief=<D_{miss}/D_{hit}>$$

Along x2, near hit and near miss about the same (near hit more) → bad feature

Along x1, near hit -> similar, and Near Miss → **very different** →**good feature**

```
Relief(S, m, τ)
    Separate S into S+ = {positive instances} and
        S- = {negative instances}
    W = (0, 0, . . . , 0)
    For i = 1 to m
        Pick at random an instance X ∈ S
        Pick at random one of the positive instances
            closest to X, Z+ ∈ S+
        Pick at random one of the negative instances
            closest to X, Z- ∈ S-
        if (X is a positive instance)
            then   Near-hit = Z+; Near-miss = Z-
            else   Near-hit = Z-; Near-miss = Z+
        update-weight(W, X, Near-hit, Near-miss)
    Relevance = (1/m)W
    For i = 1 to p
        if (relevance_i ≥ t)
            then   f_i is a relevant feature
            else   f_i is an irrelevant feature

update-weight(W, X, Near-hit, Near-miss)
    For i = 1 to p
        W_i = W_i - diff(x_i, near-hit_i)^2 + diff(x_i, near-miss_i)^2
```

Relief

- Weight update:

When x_k and y_k are nominal,

$$\text{diff}(x_k, y_k) = \begin{cases} 0 & \text{<if } x_k \text{ and } y_k \text{ are the same>} \\ 1 & \text{<if } x_k \text{ and } y_k \text{ are different>} \end{cases}$$

When x_k and y_k are numerical,

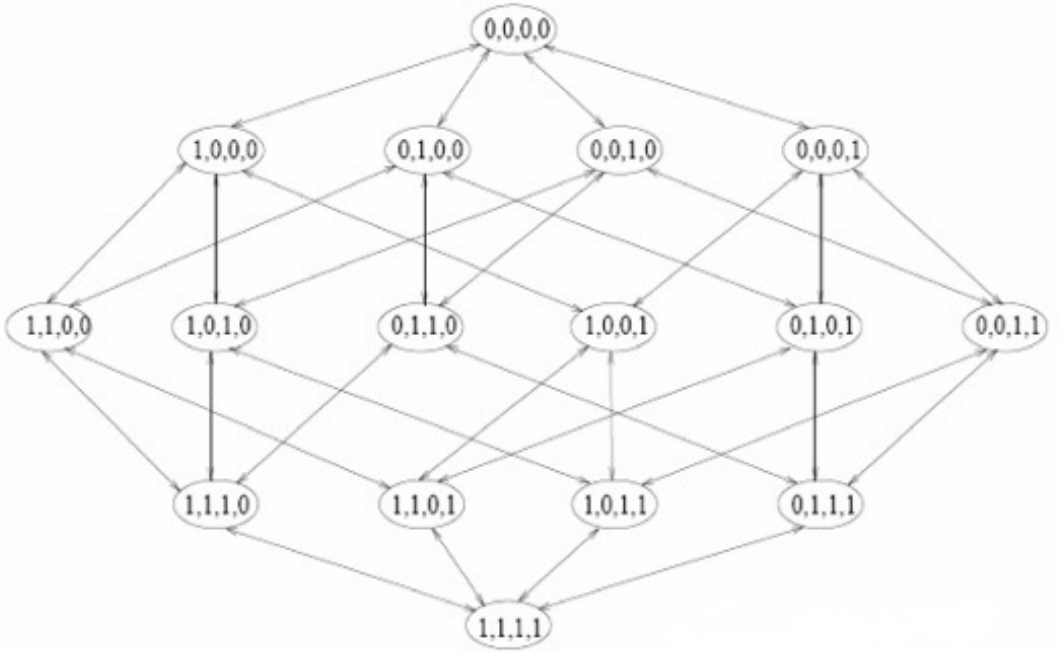
$$\text{diff}(x_k, y_k) = (x_k - y_k)/nu_k$$

where nu_k is a normalization unit to normalize the values of diff into the interval [0, 1]

WRAPPER METHODS

Wrapper Methods

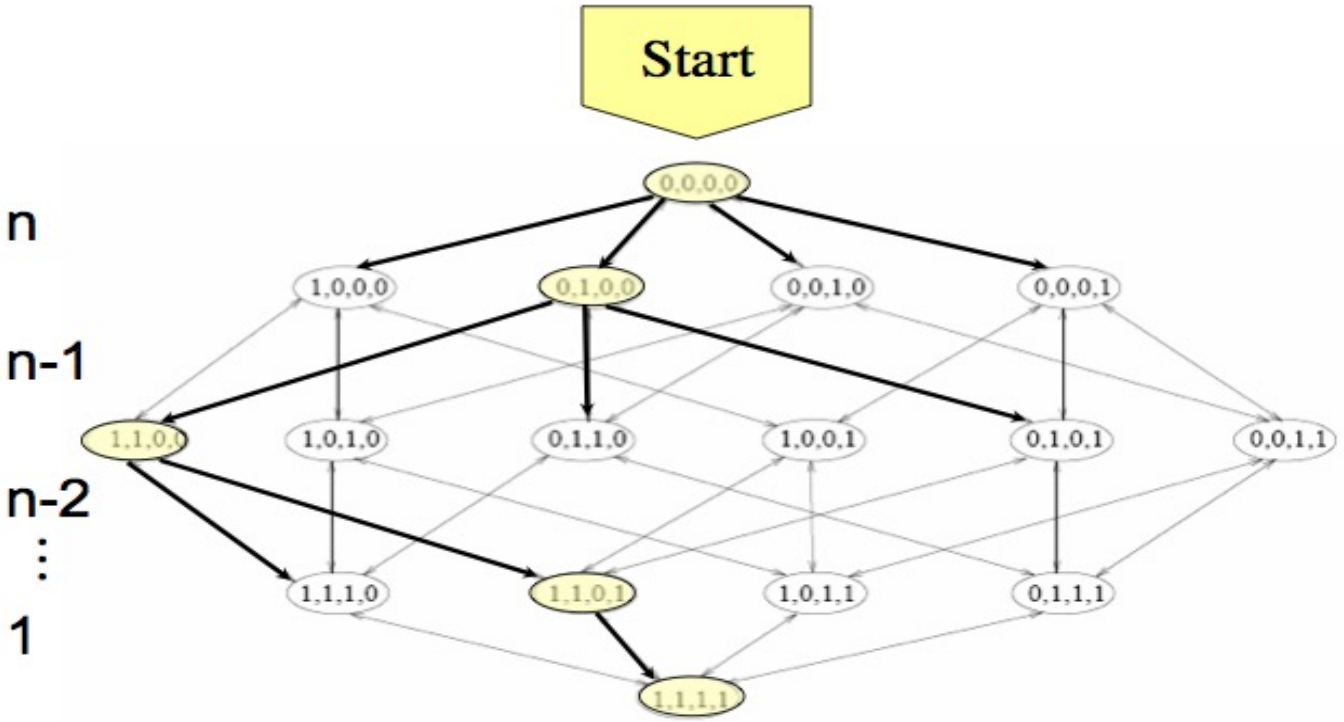
- Try subsets of features
- Evaluate using model
- Problem:
  - 2^N possible subsets!



- Search Strategies
  - Forward selection
  - Backward elimination
  - Greedy / heuristic search

Forward Selection

- Start with empty set
- Add best feature step-by-step

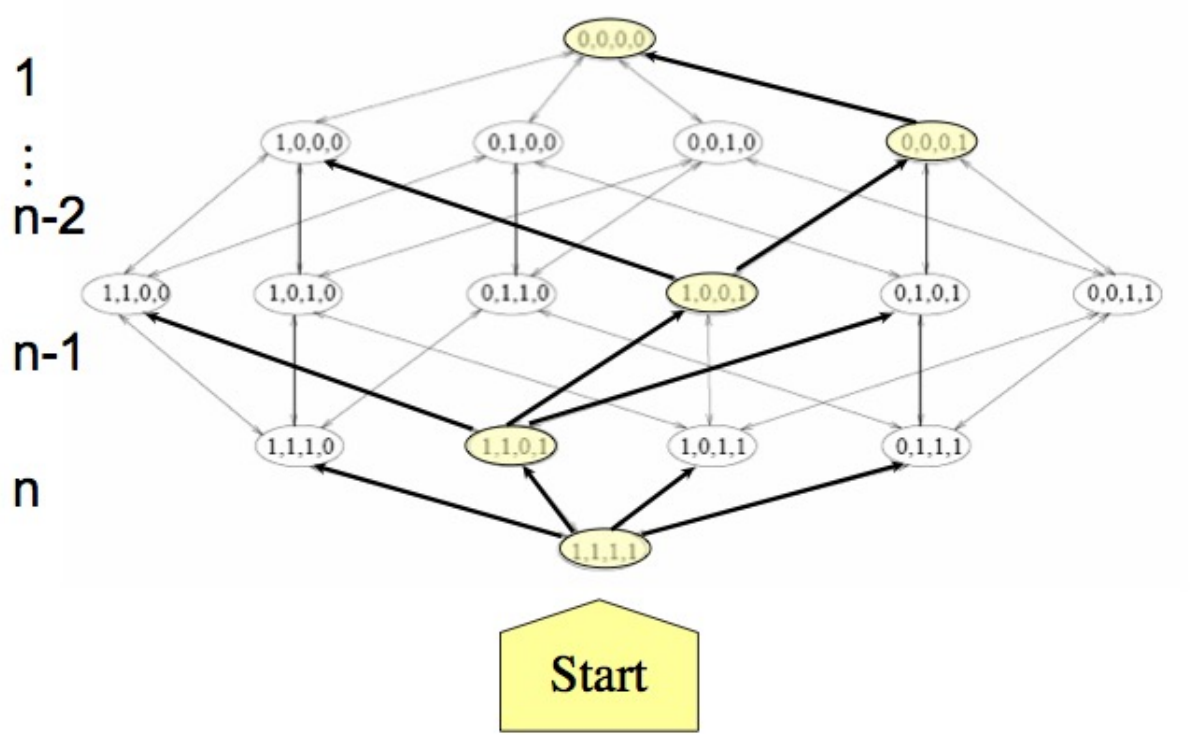


Also referred to as SFS: Sequential Forward Selection

Backward Elimination

- Start with all features
- Remove worst features

Also referred to as SBS: Sequential Backward Selection

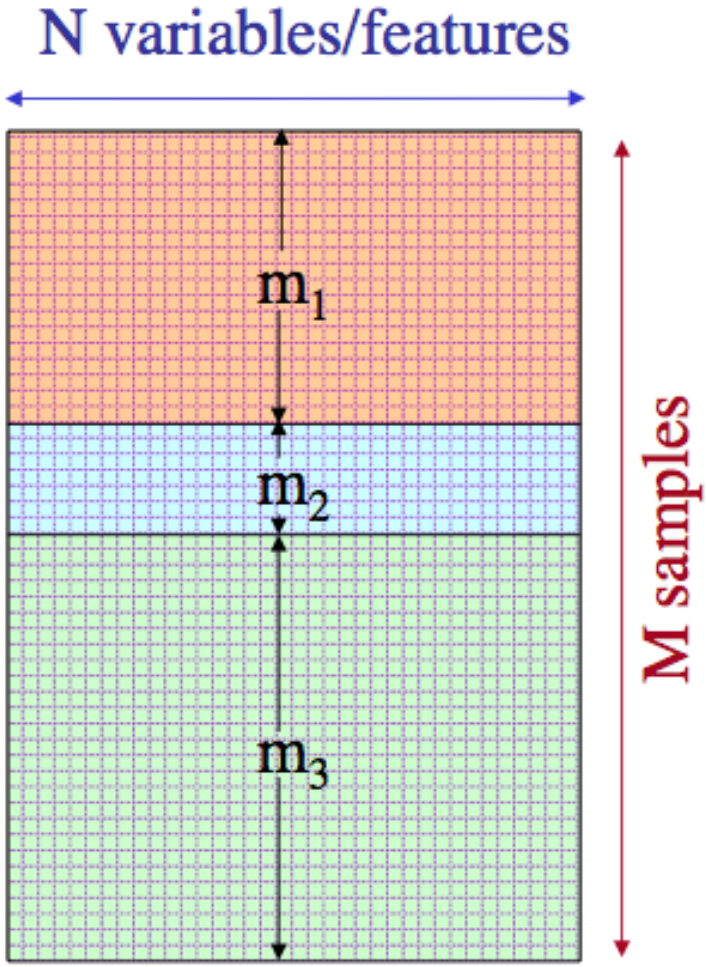


Recursive Feature Elimination (RFE)

- Steps:
  - Train model (e.g., Logistic Regression)
  - Rank features by importance (coefficients)
  - Remove weakest feature
  - Repeat

- Other training models:
  - SVM → use weights
  - Tree → use feature importance

Feature Subset Evaluation



Split data into 3 sets:

training, validation, and test set.

1) For each feature subset, train predictor on training data.

2) Select the feature subset, which performs best on validation data.

   – Repeat and average if you want to reduce variance (cross-validation).

3) Test on test data.

EMBEDDED METHODS

Embedded Methods

- Feature selection during training
- Decision Trees:
  - Select features based on:
    - entropy reduction
    - information gain

L1 Regularization

- From previous lectures
  - L1 → drives weights to zero
  - Result:
    - automatic feature selection
- This has very important practical usage

Practical Workflow

1. Remove low variance
2. Apply filter (top k features)
3. Train model (L1 / tree)
4. Cross-validation
5. Evaluate

Feature Selection in Practice (sklearn)

- VarianceThreshold
  - from sklearn.feature_selection import VarianceThreshold
- SelectKBest
  - from sklearn.feature_selection import SelectKBest
  - chi-square → chi2, mutual info → mutual_info_classif
- Recursive Feature Elimination (REF) Wrapper
  - from sklearn.feature_selection import RFE
- Lasso (Embedded) for regression
  - from sklearn.linear_model import Lasso
- LogisticRegression(penalty="l1", solver="liblinear") → for classification